



Measuring Complexity (7.1)

COT 4210 Discrete Structures II
Summer 2025
Department of Computer Science
Dr. Steinberg

Complexity

- Consider a Turing machine that decides the language $\{0^k1^k\}$. How long does it take to do it?
- Let's try a naïve version. **On input string s :**
 1. Scan across the tape and *reject* if a 0 is found to the right of a 1.
 2. Repeat while both 0s and 1s are on the tape:
 - Scan across the tape, crossing off a single 0 and a single 1.
 3. If 0s still remain after the 1s have been crossed off, or 1s after the 0s, reject. Otherwise, accept.
- This will clearly get the job done – but how good a job will it do? Let's analyze it – but first, we need the tools to do so.

Definition: Complexity

The **time complexity**, or **running time**, of a deterministic Turing machine M is a function

$$f: \mathbb{N} \rightarrow \mathbb{N}$$

where

$f(n)$ is the **maximum** number of steps that M uses on **any** input of length n .

The following three statements are equivalent:

- $f(n)$ is the running time of M .
- M runs in time $f(n)$.
- M is an $f(n)$ time Turing machine.

The Growth of Complexity

- No matter how messy an algorithm is, if you're running it on a single data item, it's probably going to be quick.
 - (There are exceptions even to that, of course.)
- We care about how fast an algorithm is in specific instances. But we care *more* about how that algorithm reacts to the *growth of data presented to it*.
 - You don't need to worry much about efficiency at $n = 10$.
 - What about at $n = 10,000$? $10,000,000$? $10,000,000,000,000$?
 - If the algorithm takes linear time to run – that is, if it runs in some constant factor of n – then the second takes about a thousand times as long as the first, and the third takes about a billion times as long.
 - What happens if the growth is quadratic – or heaven help us, exponential?
- When an algorithm is going to be used on arbitrarily-sized data sets, the most important performance factor is how it reacts to the growth of the input in these general terms. **Unlike some observations in this course, this fact is true *both* theoretically *and* practically.**

The Asymptotic Upper Bound

Let f and g be functions $f, g: \mathbb{N} \rightarrow \mathbb{R}^+$. We write:

$$f(n) = O(g(n))$$

if there are positive integers c and n_0 so that for every integer $n \geq n_0$,

$$f(n) \leq c \times g(n).$$

Uses and Limitations of Big-*O*

Let f and g be functions $f, g: \mathbb{N} \rightarrow \mathbb{R}^+$. We write:

$$f(n) = O(g(n))$$

if there are positive integers c and n_0 so that for every integer $n \geq n_0$,

$$f(n) \leq c \times g(n).$$

- Big-*O* suppresses *all* coefficients and *all* non-dominant terms. This function is $O(n^2)$:

$$f(n) = 12n^2 + 1,342,963n + 5 \times 10^{20}$$

That is intentional!

- **Asymptotic analysis is not intended to be the only type of performance analysis you use. Don't try to use it that way. It won't work.**
- It focuses on one thing, and one thing only: the dominant component of an algorithm's growth in running time versus arbitrary growth in input.
- It **will not help you** and **is not meant to help you** optimize the implementation of an algorithm. It is there to help you **pick the right algorithm in the first place.**

The Asymptotic Upper Bound

Let f and g be functions $f, g: \mathbb{N} \rightarrow \mathbb{R}^+$. We write:

$$f(n) = O(g(n))$$

Choosing the right
algorithm

if there are positive integers c and n_0 so that for every integer $n \geq n_0$,

$$f(n) \leq c \times g(n).$$

Writing good code

Some Big Classes of Big- O

- **Logarithmic:** $O(\log n)$ (lots of unit operations)
- **Linear:** $O(n)$ (really easy stuff)
- **Linearithmic or Loglinear:** $O(n \log n)$ (good sorting algorithms)
- **Quadratic:** $O(n^2)$ (bad sorting algorithms)
- **Polynomial:** $O(n^k)$ for some k (includes all of the above!)
- **Exponential:** $O(2^{n^k})$ for some k (frequently, but **not** always, 1)

(Since changing the base of a logarithm changes the result by a constant factor - remember, $\log_b n = \log_2 n / \log_2 b$ - we don't worry about logarithmic bases in asymptotic notation.)

$O(n!)$

$O(n^n)$

$O(\text{no.})$

$O(\ominus)$

Analyzing Our Algorithm

- **On input string s :**

1. Scan across the tape and *reject* if a 0 is found to the right of a 1.
2. Repeat while both 0s and 1s are on the tape:
 - Scan across the tape, crossing off a single 0 and a single 1.
3. If 0s still remain after the 1s have been crossed off, or 1s after the 0s, reject. Otherwise, accept.

Look at each of the stages:

- Stage 1 takes about n steps.
 - ($2n$ if we count going back across the tape to the left after it's done.)
- Stage 3 takes about n steps.
- Stage 2, on the other hand, takes longer:
 - Somewhere around n steps for each iteration, depending on how you handle the scanning.
 - There are $n/2$ iterations, assuming accepted input (which is actually the worst case).

Stage 2, and therefore the whole algorithm, is $O(n^2)$.

Time Complexity Classes

Consider a function:

$$t: \mathbb{N} \rightarrow \mathbb{R}^+$$

The **time complexity class**:

$$TIME(t(n))$$

is the collection of all the languages that are decidable by an $O(t(n))$ -time Turing machine.

Improving Our Algorithm

So now we know that $\{0^k1^k\}$ is in $\text{TIME}(n^2)$. Can we do better than that? Yes.

On input string s :

1. Scan across the tape and *reject* if a 0 is found to the right of a 1.
2. Repeat while both 0s and 1s are on the tape:
 - Scan across the tape, checking whether the total number of 0s and 1s remaining is even or odd.
 - If it is odd, reject.
 - Scan again across the tape, crossing off:
 - Every other 0, starting with the first.
 - Every other 1, starting with the first.
3. If no 0s or 1s remain on the tape, accept. Otherwise, reject.

Improving Our Algorithm

So now we know that $\{0^k 1^k\}$ is in $\text{TIME}(n^2)$. Can we do better than that? Yes.

On input string s :

1. Scan across the tape and *reject* if a 0 is found to the right of a 1.
2. Repeat while both 0s and 1s are on the tape:
 - Scan across the tape, checking whether the total number of 0s and 1s remaining is even or odd.
 - If it is odd, reject.
 - Scan again across the tape, crossing off:
 - Every other 0, starting with the first.
 - Every other 1, starting with the first.
3. If no 0s or 1s remain on the tape, accept. Otherwise, reject.

First, correctness. Each pass, we're:

- Checking evenness/oddness of 1s and 0s
- Dividing the number of 0s and 1s by 2
- Throwing away the remainder

This is just bitwise binary comparison!

Second, performance. We're dealing with half the input every time we scan across the tape, so we're scanning across the tape **$\log n$** times.

It takes n time to scan across the tape, so we have $O(n \log n)$.

This is actually the best we can do with a single-tape Turing machine, which brings up an interesting issue.

Complexity: Multi-Tape Turing Machines

Differing computational models don't vary in terms of what they can compute. They *do* vary in terms of how fast they can compute it.

Let's look at how we simulate multi-tape Turing machines on single-tape TMs.

Complexity: Multi-Tape Turing Machines

All the setup is clearly $O(n)$. The interesting part is the movement simulation.

- Say the MTTM takes time $t(n)$ on n . For every MTTM move, we scan the entire tape twice.
- There are at most $t(n)$ used cells per sub-tape.
- But **there are a constant number of subtapes**.
- So per-move time is all $O(t(n))$.

So we have $O(t(n)) \times t^2(n) = O(t^2(n))$.

Every $t(n)$ time multi-tape Turing machine has an equivalent $O(t^2(n))$ time Turing machine.

- Let $\#$ be a tape separator symbol.
- Add a “dotted” version of every tape alphabet symbol to the tape alphabet.
- Store the multiple tapes one after the other on our single tape, with $\#$ between each one and dotted symbols indicating the locations of the heads.
- So the tape starts out as $\#w_1 \dots w_k \# \dot{\square} \# \dot{\square} \# \dots \#$
- On each move, we:
 - Scan to find the $\#$ to indicate where each tape is, along with the dotted symbols
 - Simulate the movement of the respective head by changing the location of the dot
 - If we move right onto a $\#$, we move *everything else on the tape one space to the right*
 - Simulate the other aspects of the transition as normal for the respective single-tape Turing machine

Complexity: Nondeterministic Turing Machines

Nondeterministic TMs lead to a much more interesting result.

- Again, what we're doing here is exploring each branch of the nondeterministic computation.
- The total number of leaves in that tree is at most $b^{t(n)}$, where b is the maximum number of ND choices and $t(n)$ is the time it takes the NDTM to run. We can just call that $O(b^{t(n)})$.
- We need to run the TM once **for each of** those leaves. That's just $O(t(n))$.
- So we have $O(b^{t(n)}) \times O(t(n)) = O(t(n)b^{t(n)}) = 2^{O(t(n))}$.
- We're simulating with a 3-tape DTM, but that only squares the result, and $2^{2O(t(n))} = 2^{O(t(n))}$.

On input w :

1. Initially, tape 1 contains w ; 2 and 3 are empty.
2. Set tape 3 to 1.
3. Copy tape 1 to tape 2.
4. Use tape 2 to simulate the NDTM with w on the branch of its nondeterminism represented by tape 3.
 - a. Before each step, consult the next symbol on tape 3.
 - b. If no more symbols remain on tape 3, or the symbol value is larger than the number of choices we have, go on to step 5.
 - c. If we reach a rejecting configuration, go to step 5.
 - d. If we reach an accepting configuration, **accept and halt**.
5. Replace the string on tape 3 with the next string in the order. Go to step 3.

What We've Learned

- We've developed a basic understanding of:
 - Formal Languages and Automata Theory
 - Computability
 - Complexity
 - Time classes
- We've observed that:
 - MTTMs take polynomially longer than standard DTMs.
 - **NDTMs take *exponentially* longer than standard DTMs.**
- That last result, as it turns out, is really, really important.



Acknowledgement

Some Notes and content come from Dr. Gerber, Dr. Hughes, and Mr. Guha's COT4210 class and the Sipser Textbook, *Introduction to the Theory of Computation*, 3rd ed., 2013